Lab Procedure
Radar

Introduction

Ensure the following:

1. You have reviewed the **Application Guide – Radar**
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Explore Radar Envelope Data

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **`File > Open Folder`**..., and browse to the directory containing the python files and lab procedure (…/sensors_trainer/1_fundamentals/radar/hardware/python). Open this directory.
2. Place the Sensor Trainer upright on a table with the radar facing forward into an open area.
3. Open **radar_envelope.py** and run it using the ▷ button or the terminal. It might fail the first or second time, try running it again, it should fix itself. This will open a scoping window labeled **Radar Envelope**, showing the distances in the x-axis and amplitudes in the y-axis.
4. Move your hand toward the radar, try going from around 50 cm towards 20 cm away from the radar sensor, and observe how the envelope graph responds.
5. Set up a tape measure from the front of the device so you can notice how far you are placing the objects.
6. Collect several different objects (e.g., hand, plastic bottle, metal bottle, cardboard, notebook) and place them one at a time 30 cm away from the radar. For each object, record the behaviour of the radar and peak amplitude. You can adjust the **yLim** argument for the **BatchScope** object in the Setup display region to better view the peak amplitude.

7. Choose your most reflective object (usually metal) and move it along the tape measure from 20 cm to 60 cm. How does the peak amplitude change with distance? Record your observations.

8. Keep this object at 30 cm and move it along the 30-cm arc in front of the radar. Stop when the peak amplitude dropped below 50% of its maximum. Record the leftmost and rightmost positions and derive the FOV of the radar.

9. Attach the aluminum cone provided with the Sensors Board to the radar socket and repeat steps 6-8. How does the aluminum cone affect the radar signal regarding the peak amplitude and FOV? Record your observations.

10. Place your most reflective object around 30cm in front of the radar and hold a piece of cardboard between the radar and the object. Can you see the peak corresponding to your object? How does the cardboard affect the amplitude of the peak?

11. Gradually tilt the cardboard, from vertical to horizontal. How does the tilt angle impact the amplitude of the peak? Record your observations.

12. Stop the script using **`Ctrl+C`** and press enter when prompted.

Explore Radar IQ Mode

In the previous section, the data we saw on the scope was the output of the "envelope" mode of radar chip, which is already heavily filtered and processed. In this section we explore In-phase and Quadrature (IQ) mode, which provides more unprocessed radar data.

13. Make sure the sensor trainer is standing upright with the aluminum cone attached.

14. Open radar_IQ.py and run it using the ▷ button or the terminal. This will open a scoping window labeled **Radar IQ** showing in-phase and quadrature curves. Note that when you start zooming out of the scope, a little **A** (🅐) appears at the bottom left of the window, click it to size the data to the screen.

15. Move your hand in front of the radar around 20-25 cm away from the sensor and observe how the curves behave. Stop the script using **`Ctrl+C`** and press enter when prompted.

16. Navigate to the main radar loop region and find IQ processing section. Recall that the radar reflection signal $x[d]$ received in IQ mode is

$$x[d] = A[d]\big(\cos(\phi[d]) + j\sin(\phi[d])\big) = I[d] + jQ[d]$$

Where $\boldsymbol{I[d]}$ and $\boldsymbol{Q[d]}$ are the I and Q signals. Use the equations in concept review (025_radar) to compute amplitude and phase from IQ signals and complete this section and convert the IQ arrays to amplitude array, $\boldsymbol{A[d]}$, and phase array, $\boldsymbol{\phi[d]}$.

17. Navigate to the region 'set up experiment parameters' and uncomment the remainder of the region on instantiating amplitude and phase plots. This starts with `# scopeAmp = BatchScope`

18. Navigate to the plot to scope section inside the main radar loop region, uncomment the remainder of the section on sending data to the new plots. Then in the thread starting section towards the end of the script, uncomment the **refresh** method for amplitude and phase scope.

19. Run the script, in addition to **Radar IQ**, two extra plots named **Radar Amplitude** and **Radar Phase** will open. The amplitude plot should resemble the envelope plot in the previous activity. Put an object in front of the radar, can you still determine its distance?

20. The phase information is the key advantage of IQ mode, as it can be used to detect micro-motion. However its application is out of the scope for this lab, you can learn more about it in the radar's documentation. Stop the script using **`Ctrl+C`** and press enter when prompted.

21. Because the IQ-derived amplitude and phase are noisy, you will now implement and apply an exponential smoothing filter. The smoothing filter balances past and present data to provide a more stable signal:

$$s_t = \alpha x_t + (1 - \alpha) s_{t-1}$$

    Where s_t is the smoothed signal, x_t is the input signal, s_{t-1} is the smoothed signal from the previous time step, and α is the smoothing factor ($0 < \alpha < 1$). Navigate to the **ExpSmoother** class in the helper function region, complete the **update** method, using the previous smoothed signal, **self.prev**, and the smoothing factor, **self.alpha**.

22. Run the script again and observe all 3 plots. Does the exponential smoothing filter improve the stability of the signal? Adjust the smoothing factor, **`alpha`**, and observe its impact on the filtered signal and record your observations.

23. Stop the script using **`Ctrl+C`** and press enter when prompted.

Presence Detection

24. One simple application enabled by radar is a presence detector, where the application will trigger when an object is present within specified ranges. Keep the cone attachment on and open radar_presence.py and navigate to the **presence_detect** function in the helper function region.

25. Implement a simple presence by first isolating the amplitude values within the desired distance range. Then, compare the maximum amplitude in that range to a chosen threshold to decide whether a target is present. Complete the function and run the script, this will open a **Radar Envelope** plot and print presence detection output in the terminal.

26. Move your hand back and forth in front of the radar, does your function correctly detect presence? Adjust the threshold in your function if needed.

27. Place a piece of cardboard between your hand and the radar, does your detector still function properly? Record your observations.

28. Stop the script using **`Ctrl+C`** and press enter when prompted

Commented [LG1]: The python files say precense_detect, make sure presence is properly typed in all versions of both files haha

29. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect the metal Cone and pack them along the base and the USB cables in the provided case.